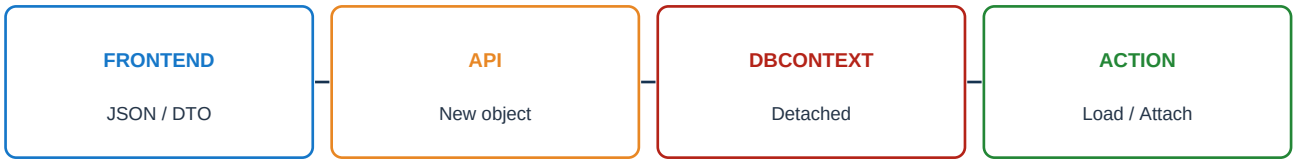
Updating Detached Entities in Real APIs

Visual Lesson Summary • Three update patterns and real API workflow

1. What Is a Detached Entity?

A **Detached** entity exists in memory, but the current **DbContext** is not tracking it. API objects commonly come from HTTP requests, mobile apps, or frontend applications—not from a query executed by the current DbContext.



Client JSON → API DTO/object → Detached entity → explicitly load or attach it before saving.

2. Pattern 1 — Load Then Update (Recommended)

```
var student = context.Students
        .First(s => s.StudentId == dto.StudentId);

student.FirstName = dto.FirstName;
student.PhoneNumber = dto.PhoneNumber;

context.SaveChanges();
```

The entity is loaded from the database and tracked automatically. Assign the fields you intend to change, then call SaveChanges(). This is recommended for most cases because it supports validation and updates detected changes.

3. Pattern 2 — Controlled Attach (Precise)

```
var student = new Student
{
        StudentId = dto.StudentId,
        PhoneNumber = dto.PhoneNumber
};

context.Attach(student);

context.Entry(student)
        .Property(s => s.PhoneNumber)
        .IsModified = true;

context.SaveChanges();
```

Attach() starts tracking the entity, but does not itself mark its properties as modified. Explicitly mark only the property you want to update. Useful for precise field updates such as PATCH-style operations.

4. Pattern 3 — Update() (Full Update Intent)

```
var student = new Student
{
        StudentId = dto.StudentId,
        FirstName = dto.FirstName,
        PhoneNumber = dto.PhoneNumber
};

context.Students.Update(student);
context.SaveChanges();
```

The lesson presents Update() as marking the entity's properties as Modified. With an incomplete DTO, fields not supplied may be written with default or null values, potentially overwriting existing data.

> **Important risk**
> A DTO containing only PhoneNumber may omit Email, Status, or RegisteredAt. Using Update() with an incomplete entity can overwrite existing database values for fields not supplied.

5. Compare the Three Patterns

Pattern	Behavior	Best fit / caution
Load then update	Load tracked row, assign selected values, save.	Recommended for most cases.
Controlled Attach	Attach stub entity; mark selected properties Modified.	Precise; developer must choose fields carefully.
Update()	Mark entity for update, then save.	Risky with incomplete DTOs.

6. Real API Example — Load Then Update

```
[HttpPut("{id}")]
public IActionResult UpdateStudent(int id, UpdateStudentDto dto)
{
        var student = context.Students
                .FirstOrDefault(s => s.StudentId == id);

        if (student == null)
                return NotFound();

        student.FirstName = dto.FirstName;
        student.PhoneNumber = dto.PhoneNumber;

        context.SaveChanges();
        return NoContent();
}
```

Check that the record exists before updating. Validate incoming data rather than trusting the client blindly.

7. Entity State and Common Mistakes

```
context.Entry(student).State
```

A detached object is not automatically updated by SaveChanges(). Load it or attach it and mark the intended properties.

• Calling SaveChanges() on an object that remains Detached.

• Using Update() with a partial DTO and overwriting missing fields.

• Trusting client values without validation.

• Ignoring the NotFound case.

8. Final Mental Model

Load → **Modify** → **Save** = recommended for most cases.
Attach → **Mark selected fields** → **Save** = controlled update.
Update() → **Full update intent** = risky when the object is incomplete.